# Penetration Testing Report: Project Neighbour

## 1. Executive Summary

A security assessment was performed on the web application deployed within the **Neighbour** laboratory environment. The evaluation targeted an authentication and authorization mechanism to determine if user data isolation controls could be bypassed.

The assessment revealed a **Critical-severity** Insecure Direct Object Reference (IDOR) vulnerability. This flaw allows an authenticated low-privileged user to access sensitive administrative profile details and capture unauthorized system flags by simply manipulating parameters in the application's URL path.

## 2. Assessment Overview & Scope

| Attribute | Details |
|---|---|
| **Target Machine IP** | MACHINE_IP (Lab Environment) |
| **Vulnerability Class** | Broken Object Level Authorization (IDOR) / OWASP A01:2021 |
| **Testing Methodology** | Black-box / Grey-box Web Application Penetration Testing |
| **Primary Risk Impact** | Information Disclosure & Account Takeover |

## 3. Technical Walkthrough & Exploitation Steps

### Phase 1: Information Gathering & Reconnaissance

Initial inspection of the target web server home directory revealed a standardized user authentication form.

1. **Source Code Inspection:** Inspecting the underlying page source layout (Ctrl + U or viewing developer tools) exposed developer documentation left intact inside HTML comments.

2. **Credential Leakage:** The comments explicitly leaked standard testing or guest-tier account credentials:
3. HTML

<!-- Use guest:guest for testing purposes -->
4.

### Phase 2: Vulnerability Exploitation (IDOR)

Using the discovered credentials, an active session was initiated.

1. **Authentication:** Authenticated successfully using the user handle guest and password guest.
2. **Parameter Analysis:** Upon a successful redirect, the application loaded the profile overview page. The address bar explicitly structure user sessions using a clear, predictable URL parameter query string:
3. HTTP

***http://MACHINE_IP/profile.php?user=guest***
4.
5. **Parameter Manipulation:** The user parameter string was manually altered from guest to target alternative administrative or common system user profiles.
6. **Data Exfiltration:** Modifying the URL query string directly to target the administrator account granted direct access to unauthorized system fields without requiring separate administrative credentials:
7. HTTP

***http://MACHINE_IP/profile.php?user=admin***
8.

**Result:** The system accepted the manual input parameter change without verifying whether the active session identifier (guest) possessed the explicit authorization rights required to view the requested resource (admin), displaying the hidden flag on screen.

# 4. Vulnerability Deep Dive

## VULN-01: Insecure Direct Object Reference (IDOR)

- **Severity:** Critical (CVSS v3.1 Base Score: **8.6** | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:N/A:N)
- **Description:** The web application references direct internal objects (user account records) via user-supplied input names (?user=) within URL routes. Because the server lacks server-side authorization enforcement checks, changing this value completely bypasses user isolation walls.

## 5. Remediation & Hardening Guidelines

To properly resolve this architectural flaw and protect user accounts, implement the following security implementations:

- **Enforce Server-Side Session Validation:** The web server must validate that the user identifier requested via parameters match the identity stored within the server's signed session token (e.g., matching the user's active session cookie or JWT payload).
- **Adopt Indirect Object References (UUIDs):** Instead of exposing predictable names or sequential database IDs directly in URLs, utilize cryptographically secure, non-sequential tokens or random UUIDs (e.g., ?user=9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d).
- **Sanitize Source Code:** Establish an automated build pipeline check to strip testing accounts and verbose HTML development comments before moving applications into live production networks.